

Approximation

Recap

Floating point numbers can be tricky sometimes because of how computers work

```
1 x = 0
2 for i in range(10):
3     x += 0.1
4 print(x == 1)
5 print(x, '=', 10*0.1)
```

Exercise

Assume you are given a string variable named `my_str`.

Write code that prints out a new string containing the even indexed characters of `my_str`.

Reminders:

- to index a string, use `string or string variable[index]`
- you can make an empty string with `string variable name = ''`
- and you can add letters to that string through `string variable name += 'letter'` or `string variable name = string variable name + 'letter'`
- range has three parameters, `range(start, stop, step)`

Decimal to Binary Algorithm

Decimal to Binary

As a reminder

$$19_{10} = 1 * 10^1 + 9 * 10^0 = 1 * 2^4 + 0 * 2^3 + 0 * 2^2 + 1 * 2^1 + 1 * 2^0 = 10011_2$$

And if we take the remainder of x `x % 2`, that gives us ____

Then if we integer divide x by 2 `x // 2`, all the bits shift right

Do this again, and again, until `x is 0`, or while `x > 0`

Decimal to Binary

```
1  num = 1507
2  result = ''
3
4  if num == 0:
5      result = '0'
6  while num > 0:
7      result = str(num % 2) + result
8      num = num // 2
```

Decimal to Binary

```
1  if num < 0:
2      is_negative = True
3      num = abs(num)
4  else:
5      is_negative = False
6
7  result = ''
8  if num == 0:
9      result = '0'
10 while num > 0:
11     result = str(num % 2) + result
12     num = num // 2
13
14 if is_negative:
15     result = "-" + result
```

Decimal to Binary

```
1  if num < 0:
2      is_negative = True
3      num = abs(num)
4  else:
5      is_negative = False
6
7  result = ''
8  if num == 0:
9      result = '0'
10 while num > 0:
11     result = str(num % 2) + result
12     num = num // 2
13
14 if is_negative:
15     result = "-" + result
```

Decimal to Binary

```
1  if num < 0:
2      is_negative = True
3      num = abs(num)
4  else:
5      is_negative = False
6
7  result = ''
8  if num == 0:
9      result = '0'
10 while num > 0:
11     result = str(num % 2) + result
12     num = num // 2
13
14 if is_negative:
15     result = "-" + result
```

Decimal to Binary

```
1  if num < 0:
2      is_negative = True
3      num = abs(num)
4  else:
5      is_negative = False
6
7  result = ''
8  if num == 0:
9      result = '0'
10 while num > 0:
11     result = str(num % 2) + result
12     num = num // 2
13
14 if is_negative:
15     result = "-" + result
```

Decimal to Binary

```
1  if num < 0:
2      is_negative = True
3      num = abs(num)
4  else:
5      is_negative = False
6
7  result = ''
8  if num == 0:
9      result = '0'
10 while num > 0:
11     result = str(num % 2) + result
12     num = num // 2
13
14 if is_negative:
15     result = "-" + result
```

Decimal to Binary

```
1  if num < 0:
2      is_negative = True
3      num = abs(num)
4  else:
5      is_negative = False
6
7  result = ''
8  if num == 0:
9      result = '0'
10 while num > 0:
11     result = str(num % 2) + result
12     num = num // 2
13
14 if is_negative:
15     result = "-" + result
```

Decimal to Binary

```
1  if num < 0:
2      is_negative = True
3      num = abs(num)
4  else:
5      is_negative = False
6
7  result = ''
8  if num == 0:
9      result = '0'
10 while num > 0:
11     result = str(num % 2) + result
12     num = num // 2
13
14 if is_negative:
15     result = "-" + result
```

Definition of fractions in number systems

If $19_{10} = 10 * 10^1 + 9 * 10^0$, then what is 0.19_{10} ?

Definition of fractions in number systems

The fraction $0.abc$ means

$$a * 10^{-1} + b * 10^{-2} + c * 10^{-3}$$

And for binary representation, we use the same idea

$$a * 2^{-1} + b * 2^{-2} + c * 2^{-3}$$

Or in sentence form

The binary representation of a decimal fraction f would require finding the values of a , b , c , etc. such that

$$f = 0.5a + 0.25b + 0.125c + 0.0625d + \dots$$

Because $2^{-1} = 0.5$, and $2^{-2} = 0.25$, and $2^{-3} = 0.125$, etc.

Decimal fraction to binary fraction

Given that you know how to convert whole numbers to binary, how would you convert a

$3/8$ to a binary

$$\frac{3}{8} = 0.375_{10} = 3 * 10^{-1} + 7 * 10^{-2} + 5 * 10^{-3}$$

A trick to convert to a decimal fraction to binary is to multiply it by a big enough power of 2 to turn it into a whole number

Has to be a power of 2 because we are converting to binary

Decimal fraction to binary fraction

Given

$$3/8 = 0.375_{10}$$

1. $0.375 * 2^1 = 0.75$

2. $0.375 * 2^2 = 1.5$

3. $0.375 * 2^3 = 3.0$

So we multiply by 2^3 to get a whole number

Then convert that whole number to binary

$$3_{10} = 11_2$$

Decimal fraction to binary fraction

Then divide it by 2^3 to get the binary fraction, shifting the bits to the right by 3 places

$$11_2 / 2^3 = 0.011_2$$

Because dividing by some power of 2 means shifting the bits to the right by that many places

Like how dividing by 10 means shifting the decimal point to the left by that many places

But

What if we *can't* find a power of 2 that turns it into a whole number?

This program relies on the fact that multiplying by 2 will eventually turn the fraction into a whole number

This is guaranteed for any value stored as a float, because a float is already stored internally as a finite binary (dyadic) fraction — i.e. some integer divided by a power of 2. So there always exists a power of 2 large enough to shift it into a whole number, even if that power is large

This is *not* guaranteed for the "true" decimal value the float is approximating (e.g. the real number 0.1) — which is exactly why the "Issues" example below can require a very long, cut-off binary expansion

```
1  x = 0.625
2
3  p = 0
4  while ((2**p) * x) % 1 != 0:
5      print("Remainder =", (2**p) * x % 1)
6      p += 1
7  num = int(x * (2**p)) # is now a whole number
8
9  result = ''
10 if num == 0:
11     result = '0'
12 while num > 0:
13     result = str(num % 2) + result
14     num = num // 2
15
16 for i in range(p - len(result)):
17     result = '0' + result
18
19 if len(result) == p:
20     result = '0' + result
21
22 result = result[0:-p] + '.' + result[-p:]
23 print(result)
```



```
1  x = 0.625
2
3  p = 0
4  while ((2**p) * x) % 1 != 0:
5      print("Remainder =", (2**p) * x % 1)
6      p += 1
7  num = int(x* (2**p)) # is now a whole number
8
9  result = ''
10 if num == 0:
11     result = '0'
12 while num > 0:
13     result = str(num % 2) + result
14     num = num // 2
15
16 for i in range(p - len(result)):
17     result = '0' + result
18
19 if len(result) == p:
20     result = '0' + result
21
22 result = result[0:-p] + '.' + result[-p:]
23 print(result)
```

```
1  x = 0.625
2
3  p = 0
4  while ((2**p) * x) % 1  $\neq$  0:
5      print("Remainder =", (2**p) * x % 1)
6      p += 1
7  num = int(x* (2**p)) # is now a whole number
8
9  result = ''
10 if num == 0:
11     result = '0'
12 while num > 0:
13     result = str(num % 2) + result
14     num = num // 2
15
16 for i in range(p - len(result)):
17     result = '0' + result
18
19 if len(result) == p:
20     result = '0' + result
21
22 result = result[0:-p] + '.' + result[-p:]
23 print(result)
```

```
1  x = 0.625
2
3  p = 0
4  while ((2**p) * x) % 1  $\neq$  0:
5      print("Remainder =", (2**p) * x % 1)
6      p += 1
7  num = int(x* (2**p)) # is now a whole number
8
9  result = ''
10 if num == 0:
11     result = '0'
12 while num > 0:
13     result = str(num % 2) + result
14     num = num // 2
15
16 for i in range(p - len(result)):
17     result = '0' + result
18
19 if len(result) == p:
20     result = '0' + result
21
22 result = result[0:-p] + '.' + result[-p:]
23 print(result)
```

```
1  x = 0.625
2
3  p = 0
4  while ((2**p) * x) % 1 != 0:
5      print("Remainder =", (2**p) * x % 1)
6      p += 1
7  num = int(x * (2**p)) # is now a whole number
8
9  result = ''
10 if num == 0:
11     result = '0'
12 while num > 0:
13     result = str(num % 2) + result
14     num = num // 2
15
16 for i in range(p - len(result)):
17     result = '0' + result
18
19 if len(result) == p:
20     result = '0' + result
21
22 result = result[0:-p] + '.' + result[-p:]
23 print(result)
```

Issues

What is the output of

- 0.375
- 0.5
- 0.1

Suppose that we want to represent

- $1 * 2^{-3}$, that would be 0.001_2 because it's a power of 2
- but 0.1 is not a power of 2, so it can't be represented exactly in binary
- so the output is long, potentially infinite, and has to be cut off at some point

Computers

Everything in computers is bits, because of that some numbers are just too big to fit in the bits available

So how are these bits actually computed in computer memory?

2 values, where one is the number and the other is the exponent

- $[1, 1] \rightarrow 1 * 2^1 = 10_2 = 2.0_{10}$
- $[1, -1] \rightarrow 1 * 2^{-1} = 0.1_2 = 0.5_{10}$
- $[125, -2] \rightarrow 125 * 2^{-2} = 11111.01_2 = 31.25_{10}$

Floating decimal point

We use a finite set of bits to represent a potentially infinite set of numbers

- the max number of digits governs the precision with which numbers can be represented
- Most modern computers use 64-bit "double precision" floats, which reserve 52 bits for the significant digits (mantissa), 11 bits for the exponent, and 1 bit for the sign
- And if a number is too big to represent exactly, it is rounded to the nearest representable number, and the rounding error shows up in the last (52nd) bit of the mantissa
- because the error is relative to the mantissa, not an absolute value, it stays tiny ($\approx 2^{-52} \approx 2.2 * 10^{-16}$ of the number's own magnitude) — but it's never exactly zero, which is why repeated addition can drift away from the exact answer

But even really small errors lead to large problems

```
1 x = 0
2 for i in range(10):
3     x += 0.125
4 print(x == 1.25) # true
```

```
1 x = 0
2 for i in range(10):
3     x += 0.1
4 print(x == 1) # false
```

So

- Never use `=` to compare floating point numbers
- Instead, check if the absolute difference is smaller than some small number, like 10^{-9} otherwise known as epsilon
- we need to be really careful when making applications that uses floating point numbers

We can make a better guess and check

Through approximation, when we want to find an answer where the answer isn't an integer

Because

- Exact answers may not exist
- "good enough" answers may be sufficient

Finding the square root

- last time we made a guess and check algorithm to find the square root of a number
- but it only worked for perfect squares
- what if we wanted to find the square root of any positive integer

Question

- what does it mean to find the square root of x ?

Usually it's

- find r such that $r * r = x$
- if x is not a perfect square, then r is not an integer

Approximation

1. find an answer that's good enough

- like finding r such that $r * r$ is only a small distance away from x
- epsilon ϵ is usually the term used for "small distance away"
- where we want to find r such that $|r^2 - x| < \epsilon$

2. Algorithm

- start with a guess, g , that we know is too small (like 0)
- increment by a small value, a , to give a new guess g
- check if $g**2$ is close enough to x (within epsilon)
- continue until we get an answer that's close enough

3. looking at all possible values $g + k*a$ for integer values of k

4. stop when $|g**2 - x| < \text{epsilon}$

Approximation

1. so we have two parameters to set

- epsilon, how close are we to the answer
- increment, how much to increase our guess by

2. And performance will vary based on these values

- speed
- accuracy
- lower increment means more steps, a slower program, but higher accuracy
- higher increment means fewer steps, a faster program, but lower accuracy, and may skip over the answer
- lower epsilon means more steps, a slower program, but higher accuracy
- higher epsilon means fewer steps, a faster program, but lower accuracy

Implementation

```
1  x = 36
2  epsilon = 0.01
3  num_guesses = 0
4  guess = 0.0
5  increment = 0.0001
6
7  while abs(guess**2 - x) ≥ epsilon:
8      guess += increment
9      num_guesses += 1
10
11 print("num_guesses =", num_guesses)
12 print(guess, "is close to square root of", x)
```

Implementation

```
1  x = 36
2  epsilon = 0.01
3  num_guesses = 0
4  guess = 0.0
5  increment = 0.0001
6
7  while abs(guess**2 - x) ≥ epsilon:
8      guess += increment
9      num_guesses += 1
10
11 print("num_guesses =", num_guesses)
12 print(guess, "is close to square root of", x)
```

Implementation

```
1  x = 36
2  epsilon = 0.01
3  num_guesses = 0
4  guess = 0.0
5  increment = 0.0001
6
7  while abs(guess**2 - x) ≥ epsilon:
8      guess += increment
9      num_guesses += 1
10
11 print("num_guesses =", num_guesses)
12 print(guess, "is close to square root of", x)
```

Implementation

```
1  x = 36
2  epsilon = 0.01
3  num_guesses = 0
4  guess = 0.0
5  increment = 0.0001
6
7  while abs(guess**2 - x) ≥ epsilon:
8      guess += increment
9      num_guesses += 1
10
11 print("num_guesses =", num_guesses)
12 print(guess, "is close to square root of", x)
```

Implementation

```
1  x = 36
2  epsilon = 0.01
3  num_guesses = 0
4  guess = 0.0
5  increment = 0.0001
6
7  while abs(guess**2 - x) ≥ epsilon:
8      guess += increment
9      num_guesses += 1
10
11 print("num_guesses =", num_guesses)
12 print(guess, "is close to square root of", x)
```

Implementation

```
1  x = 36
2  epsilon = 0.01
3  num_guesses = 0
4  guess = 0.0
5  increment = 0.0001
6
7  while abs(guess**2 - x) ≥ epsilon:
8      guess += increment
9      num_guesses += 1
10
11 print("num_guesses =", num_guesses)
12 print(guess, "is close to square root of", x)
```

Implementation

```
1  x = 36
2  epsilon = 0.01
3  num_guesses = 0
4  guess = 0.0
5  increment = 0.0001
6
7  while abs(guess**2 - x) ≥ epsilon:
8      guess += increment
9      num_guesses += 1
10
11 print("num_guesses =", num_guesses)
12 print(guess, "is close to square root of", x)
```

Implementation

```
1  x = 36
2  epsilon = 0.01
3  num_guesses = 0
4  guess = 0.0
5  increment = 0.0001
6
7  while abs(guess**2 - x) ≥ epsilon:
8      guess += increment
9      num_guesses += 1
10
11 print("num_guesses =", num_guesses)
12 print(guess, "is close to square root of", x)
```

Implementation

Let's try finding the square root of

- 36
- 24
- 2
- 12345
- 54321

Will this loop always terminate?

Implementation

Let's try to debug

```
1  x = 36
2  epsilon = 0.01
3  num_guesses = 0
4  guess = 0.0
5  increment = 0.0001
6
7  while abs(guess**2 - x) >= epsilon:
8      guess += increment
9      num_guesses += 1
10
11     if num_guesses % 100000 == 0:
12         print("current guess =", guess)
13         print("current guess^2 =", guess**2)
14         print("distance from x =", abs(guess**2 - x))
15     if num_guesses % 1000000 == 0:
16         input("continue?")
17
18 print("num_guesses =", num_guesses)
19 print(guess, "is close to square root of", x)
```

Implementation

Let's try to debug

```
1  x = 36
2  epsilon = 0.01
3  num_guesses = 0
4  guess = 0.0
5  increment = 0.0001
6
7  while abs(guess**2 - x) ≥ epsilon:
8      guess += increment
9      num_guesses += 1
10
11     if num_guesses % 100000 == 0:
12         print("current guess =", guess)
13         print("current guess^2 =", guess**2)
14         print("distance from x =", abs(guess**2 - x))
15     if num_guesses % 1000000 == 0:
16         input("continue?")
17
18 print("num_guesses =", num_guesses)
19 print(guess, "is close to square root of", x)
```

Implementation

Let's try to debug

```
1  x = 36
2  epsilon = 0.01
3  num_guesses = 0
4  guess = 0.0
5  increment = 0.0001
6
7  while abs(guess**2 - x) >= epsilon:
8      guess += increment
9      num_guesses += 1
10
11     if num_guesses % 100000 == 0:
12         print("current guess =", guess)
13         print("current guess^2 =", guess**2)
14         print("distance from x =", abs(guess**2 - x))
15     if num_guesses % 1000000 == 0:
16         input("continue?")
17
18 print("num_guesses =", num_guesses)
19 print(guess, "is close to square root of", x)
```

The fix

```
1  x = 36
2  epsilon = 0.01
3  num_guesses = 0
4  guess = 0.0
5  increment = 0.0001
6
7  while abs(guess**2 - x) ≥ epsilon and guess**2 ≤ x:
8      guess += increment
9      num_guesses += 1
10
11  print("num_guesses =", num_guesses)
12  if abs(guess**2 - x) ≥ epsilon:
13      print("Failed to find the square root of", x)
14  else:
15      print(guess, "is close to square root of", x)
```

The fix

```
1  x = 36
2  epsilon = 0.01
3  num_guesses = 0
4  guess = 0.0
5  increment = 0.0001
6
7  while abs(guess**2 - x) ≥ epsilon and guess**2 ≤ x:
8      guess += increment
9      num_guesses += 1
10
11  print("num_guesses =", num_guesses)
12  if abs(guess**2 - x) ≥ epsilon:
13      print("Failed to find the square root of", x)
14  else:
15      print(guess, "is close to square root of", x)
```

The fix

```
1  x = 36
2  epsilon = 0.01
3  num_guesses = 0
4  guess = 0.0
5  increment = 0.0001
6
7  while abs(guess**2 - x) ≥ epsilon and guess**2 ≤ x:
8      guess += increment
9      num_guesses += 1
10
11  print("num_guesses =", num_guesses)
12  if abs(guess**2 - x) ≥ epsilon:
13      print("Failed to find the square root of", x)
14  else:
15      print(guess, "is close to square root of", x)
```

The fix

```
1  x = 36
2  epsilon = 0.01
3  num_guesses = 0
4  guess = 0.0
5  increment = 0.0001
6
7  while abs(guess**2 - x) ≥ epsilon and guess**2 ≤ x:
8      guess += increment
9      num_guesses += 1
10
11  print("num_guesses =", num_guesses)
12  if abs(guess**2 - x) ≥ epsilon:
13      print("Failed to find the square root of", x)
14  else:
15      print(guess, "is close to square root of", x)
```

Now it stops if it overshoots and reports an error

But what if we don't want to fail? What can we do